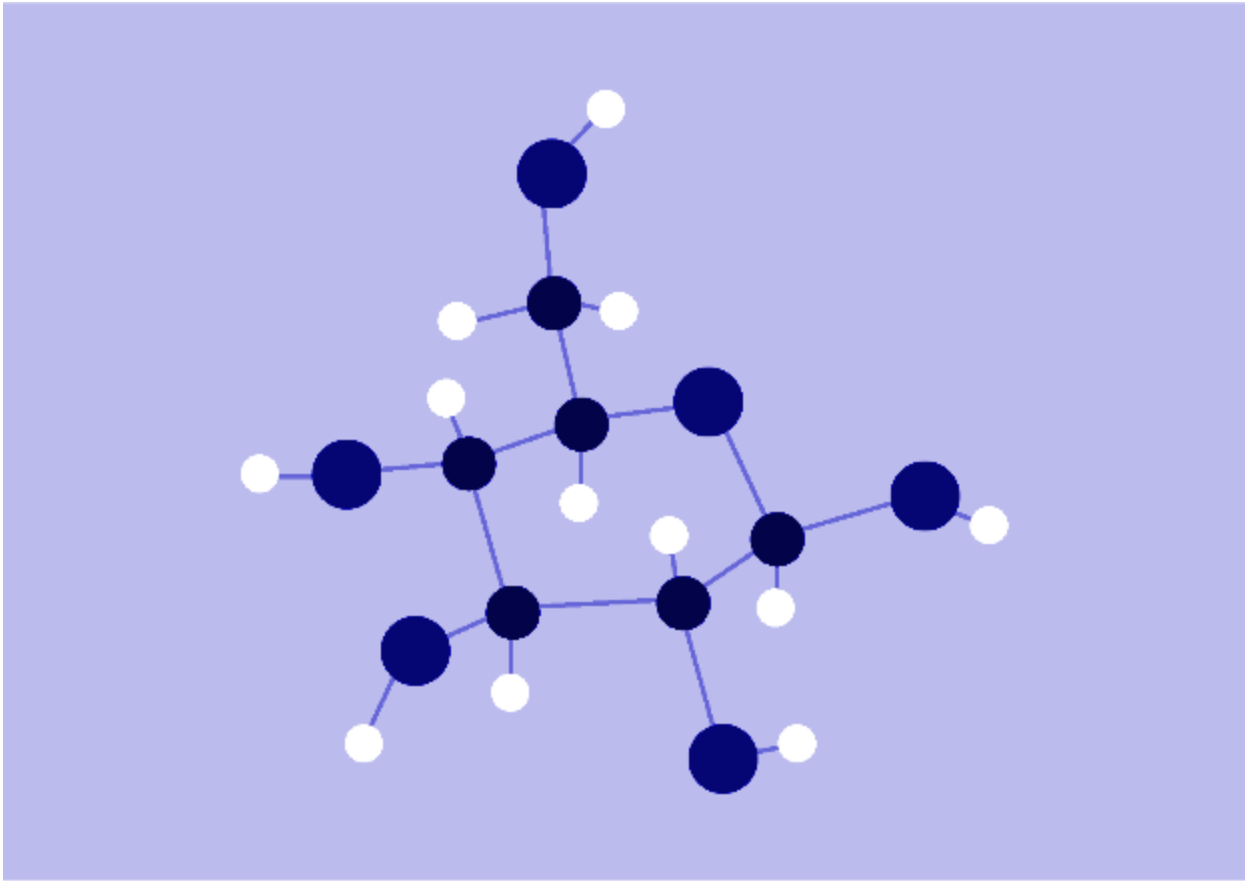
Testkonzept

ChronoScope - Online Aufgaben-Planungssystem/Selbst-Managementsystem



NI0 - Name Ideas 0

Stand 11.04.2026

Autor: Elias Kerlin

Prüfleser: Paul Berndt, Felix Weglehner

1 Einleitung **2**
1.1 Projektbeschreibung 2
1.2 Zweck des Testkonzepts 2
1.3 Reifegrad und Vorgehensweise 2
1.4 Abgrenzung 3
2 Testobjekte **3**
2.1 Backend-Komponenten 3
2.2 Frontend-Komponenten 4
3 Qualitätsmerkmale **4**
4 Testziele **5**
5 Testarten und Scope **6**
5.1 Backend 6
5.2 Frontend 11
5.3 Sicherheitsszenarien 14
5.4 Nicht behandelte Testarten 15
6 Testwerkzeuge **15**
6.1 Backend 15
6.2 Frontend 16
6.3 Testautomatisierung / CI 16
7 Teststrategie **17**
7.1 Mocking-Strategie 17
7.2 Testdaten 18
7.3 Testabdeckung 18
8 Testumgebung **19**
8.1 Lokale Entwicklungsumgebung 19
8.2 CI-Umgebung (GitHub Actions) 19
9 Testkriterien **20**
9.1 Eintrittskriterien 20
9.2 Abbruchkriterien 20
9.3 Erfolgskriterien / Abnahme 20
10 Dokumentation und Testergebnisse **21**
11 Verantwortlichkeiten **21**
12 Risiken **22**

1 Einleitung

1.1 Projektbeschreibung

ChronoScope ist ein browserbasiertes Online-Aufgabenplanungssystem, das Benutzern ermöglicht, Aufgaben zu erfassen, Abhängigkeiten zwischen Aufgaben abzubilden und Arbeitszeiten zu hinterlegen. Auf Basis dieser Informationen plant ein automatisierter Algorithmus die Aufgaben optimiert in den Kalender des Benutzers ein. Das System unterstützt mehrere Organisationen und die Zuordnung von Benutzern zu diesen (Mandantenfähigkeit). Die Authentifizierung erfolgt über Keycloak als zentralen Identity Provider.

Das System besteht aus einem Angular-Frontend, einem Spring-Boot-Backend, einer MariaDB-Datenbank sowie Keycloak und einem SMTP-Mailserver als externe Dienste.

1.2 Zweck des Testkonzepts

Dieses Dokument beschreibt die Teststrategie, den Testumfang, die eingesetzten Werkzeuge und die fachlichen Testszenarien für das Projekt ChronoScope. Es dient als verbindliche Grundlage für alle Entwickler und stellt sicher, dass die funktionalen Anforderungen aus dem Pflichtenheft durch Tests abgedeckt werden.

1.3 Reifegrad und Vorgehensweise

Das Testkonzept wird in zwei Phasen befüllt:

1. **Jetzt (vor/zu Beginn der Implementierung):** Teststrategie, Scope, Werkzeuge und fachliche Testszenarien werden spezifikationsbasiert aus Pflichtenheft und Designbeschreibung abgeleitet (Black-Box-Ansatz). Konkrete Klassennamen, Methodensignaturen und Testklassen werden noch nicht angegeben.
2. **Während der Implementierung:** Konkrete Testklassen, Methodennamen und ggf. neue Randfälle werden iterativ ergänzt, ohne die Gesamtstruktur zu verändern.

Das Dokument ist damit zu jedem Zeitpunkt vollständig strukturiert - aber nicht zwingend final mit allen Implementierungsdetails befüllt. Dieser Ansatz ist methodisch korrekt und stellt sicher, dass aus Anforderungen getestet wird, statt nur gegen den späteren Code.

1.4 Abgrenzung

Dieses Testkonzept umfasst automatisierte Unit-Tests und Integrationstests für Backend und Frontend. Externe Dienste (Keycloak, SMTP, MariaDB in Produktion) werden im Testkontext nicht direkt geprüft, sondern gemockt oder durch Alternativen ersetzt (H2 In-Memory-DB, gemockte OAuth-Services).

2 Testobjekte

2.1 Backend-Komponenten

Das Backend folgt einer dreischichtigen Architektur, ergänzt durch ein Spring-unabhängiges Algorithmus-Layer.

Komponente	**Beschreibung**
Controller-Schicht	REST-Endpunkte; Entgegennahme und Validierung von Requests, Weiterleitung an Services, Fehlerbehandlung und Response-Aufbau
Service-Schicht	Fachliche Geschäftslogik; Orchestrierung von Repository-Zugriffen und Algorithmusaufrufen
Repository-Schicht	Datenbankzugriff via Spring Data JPA; Persistenz von Aufgaben, Zeitslots, Scopes, Organisationen und Abhängigkeiten
Algorithmus-Layer (CPM / Topologische Sortierung / Planung)	Spring-unabhängige Java-Klassen; kritischer Pfad, topologische Sortierung und heuristikbasierte Aufgabeneinplanung

Konkrete Klassennamen werden in der Implementierungsphase ergänzt.

2.2 Frontend-Komponenten

Das Frontend basiert auf Angular 21 mit Standalone Components.

Komponente	Beschreibung
UI-Komponenten (Standalone Components)	Angular-Komponenten für Kalenderansicht, Listenansicht, Aufgaben-Modal, Einstellungen u. a.
Services	Zustandsverwaltung (z. B. Aufgaben-State via RxJS BehaviorSubject), HTTP-Kommunikation mit dem Backend, Authentifizierungs-Logik
Stores	Persistierter App-State (z. B. Benutzereinstellungen via Signal-basiertem Store)

Konkrete Komponentennamen werden in der Implementierungsphase ergänzt.

3 Qualitätsmerkmale

Die Teststrategie fokussiert gemäß ISO 25010 auf folgende Qualitätsmerkmale:

Merkmal	Testdurchführung
Funktionale Korrektheit	Automatisierte Unit- und Integrationstests prüfen, ob das System die spezifizierten Anforderungen des Pflichtenhefts korrekt umsetzt
Fehlertoleranz / Robustheit	Tests prüfen das Verhalten bei ungültigen Eingaben, fehlenden Pflichtfeldern und unzureichenden Kapazitäten; korrekte Fehlermeldungen und HTTP-Statuscodes werden verifiziert

Zuverlässigkeit	Integrationstests stellen sicher, dass Datenbankoperationen korrekt funktionieren und Transaktionen bei Fehlern korrekt zurückgerollt werden

Folgende Qualitätsmerkmale werden im Rahmen dieses Testkonzepts nicht durch automatisierte Tests abgedeckt:

- **Performance / Reaktionszeit** (/NFA010/): Antwortzeiten unter 5 Sekunden werden nicht durch automatisierte Lasttests geprüft. Der Zeitrahmen rechtfertigt keinen separaten Performance-Test-Aufbau. Die Anforderung wird implizit durch manuelles Testen beobachtet.
- **Sicherheit / Authentifizierung**: Sicherheitstests sind in Abschnitt 5.3 als Szenarien beschrieben und werden vollständig implementierbar, sobald die Authentifizierungspflicht aktiv erzwungen wird.
- **Benutzbarkeit / Usability** (/NFA031/): Wird durch manuelles Testen (Entwickler-Sichttests im Browser) beobachtet.

4 Testziele

#	Ziel	Messkriterium
TZ-01	Das Algorithmus-Layer (CPM, topologische Sortierung, Planung) ist durch Unit-Tests vollständig abgedeckt	Alle relevanten Zweige der Algorithmen (Normalfall, Zykluserkennung, unlösbare Planung) sind durch Tests abgedeckt
TZ-02	Die fachliche Eingabevalidierung im Backend ist durch Tests abgesichert	Ungültige Requests (fehlende Pflichtfelder, Formatverletzungen) führen zuverlässig zu 400 Bad Request
TZ-03	Der Planungs-Service verhält sich korrekt bei allen relevanten Kapazitätsszenarien	Alle Fälle (exakt passend, Überschuss, Unterdeckung, Aufteilung über Slots) sind durch Tests abgedeckt

TZ-04	Die Backend-REST-Endpunkte funktionieren im vollständigen Spring-Kontext korrekt	Integrationstest-Szenarien für alle spezifizierten Endpunkte sind vorhanden und erfolgreich
TZ-05	Frontend-Services verwalten den Applikationszustand korrekt	Zustandsübergänge (Aufgabe hinzufügen, aktualisieren, entfernen) sind durch Unit-Tests verifiziert
TZ-06	Frontend-Formularkomponenten blockieren ungültige Submissions	Pflichtfeld-Verletzungen verhindern das Absenden; gültige Eingaben lösen den Service-Aufruf aus
TZ-07	Für automatisierte Tests wird eine angemessene strukturelle Testabdeckung erreicht	Projektweit wird eine Zweigüberdeckung von mindestens 75% angestrebt; für das Algorithmus-Layer mindestens 90%

5 Testarten und Scope

5.1 Backend

5.1.1 Unit Tests - Algorithmus-Layer

Begründung: Das Algorithmus-Layer ist eine eigenständige, Spring-unabhängige Java-Implementierung. Es kann direkt und ohne Testinfrastruktur-Overhead isoliert getestet werden. Da dieses Layer die algorithmische Kernlogik des Systems enthält, hat es den höchsten Testbedarf.

Testszenarien - Topologische Sortierung (DFS):

ID	Szenario	Erwartetes Ergebnis
BE-U-01	Aufgaben ohne Abhängigkeiten	Rückgabe aller Aufgaben in beliebiger (aber konsistenter) Reihenfolge

BE-U-02	Aufgaben mit linearer Abhängigkeitskette (A → B → C)	Rückgabe in korrekter topologischer Reihenfolge (A vor B vor C)
BE-U-03	Aufgaben mit Abhängigkeitsgraph (A → C, B → C)	C erscheint nach A und B in der sortierten Liste
BE-U-04	Aufgaben mit zyklischer Abhängigkeit (A → B → A)	Zyklusausnahme wird ausgelöst

Testszenarien - Critical Path Method (CPM):

ID	Szenario	Erwartetes Ergebnis
BE-U-10	Einzelne Aufgabe ohne Abhängigkeiten	`earliestStart = Startzeit`, `slack = 0` (nur Aufgabe auf dem kritischen Pfad)
BE-U-11	Zwei parallele Aufgaben mit gemeinsamer Nachfolgeraufgabe	Aufgabe mit längerer Dauer liegt auf dem kritischen Pfad (Slack = 0), kürzere hat positiven Slack
BE-U-12	Lineare Kette von drei Aufgaben	Gesamte Kette liegt auf dem kritischen Pfad; für alle gilt Slack = 0; `earliestFinish` und `latestFinish` stimmen überein
BE-U-13	Aufgabe mit mehreren Vorgängern	`earliestStart` der Aufgabe = maximales `earliestFinish` aller Vorgänger

Testszenarien - Planungsalgorithmus:

ID	Szenario	Erwartetes Ergebnis
BE-U-20	Aufgabe mit Deadline; ausreichend Kapazität vorhanden	Aufgabe wird vollständig vor ihrer Deadline eingeplant
BE-U-21	Aufgabe mit Deadline; nicht genug Kapazität vor der Deadline	Algorithmus gibt `null` zurück (keine gültige Planung gefunden)
BE-U-22	Zwei unabhängige Aufgaben	Beide Aufgaben werden eingeplant; Scopes überlappen sich nicht
BE-U-23	Abhängige Aufgaben (A muss vor B abgeschlossen sein)	Der Scope von B beginnt nicht früher als das Ende von A
BE-U-24	Backtracking-Fall: erste Auswahl führt zu keiner Lösung	Algorithmus backtracks und findet alternative gültige Einplanung
BE-U-25	Keine Zeitslots vorhanden	Algorithmus gibt `null` zurück oder behandelt den Sonderfall definiert
BE-U-26	Keine Aufgaben vorhanden	Rückgabe einer leeren Liste von Scopes

5.1.2 Unit Tests - Service- und Controller-Schicht

Begründung: Service-Tests prüfen die fachliche Geschäftslogik isoliert von Datenbank und HTTP-Layer. Das Repository wird gemockt (Mockito). Controller-Tests prüfen die Anfrageverarbeitung, Validierung und Response-Aufbau isoliert vom Spring-Kontext (alternativ via MockMvc als leichtgewichtiger Kontext).

Testszenarien - Planungs-Service (/F160/):

ID	Szenario	Erwartetes Ergebnis

BE-U-30	Gesamtdauer Aufgaben = Gesamtdauer Slots	Planung erfolgreich; alle Aufgaben sind vollständig eingeplant
BE-U-31	Gesamtdauer Aufgaben < Gesamtdauer Slots	Planung erfolgreich; verbleibende Slot-Zeit bleibt ungenutzt
BE-U-32	Gesamtdauer Aufgaben > Gesamtdauer Slots	Fehler wird ausgelöst (unzureichende Kapazität); Fehlermeldung ist aussagekräftig
BE-U-33	Aufgabe überschreitet Slot-Ende; nächster freier Slot vorhanden	Aufgabe wird auf zwei Scopes aufgeteilt (erster Scope endet mit dem Slot, zweiter beginnt im nächsten Slot)
BE-U-34	Keine Aufgaben vorhanden	Leere Scope-Liste wird zurückgegeben; kein Fehler
BE-U-35	Keine Zeitslots vorhanden	Fehler wird ausgelöst

Testszenarien - Aufgaben-Verwaltung (/F050/, /F060/):

ID	**Szenario**	**Erwartetes Ergebnis**
BE-U-40	Aufgabe anlegen mit allen Pflichtfeldern (Titel, Dauer)	Aufgabe wird über Repository persistiert; erstellte Aufgabe wird zurückgegeben
BE-U-41	Aufgabe anlegen: Name zu kurz (< 1 Zeichen)	Validierungsfehler; keine Datenbankoperation
BE-U-42	Aufgabe anlegen: Dauer fehlt	Validierungsfehler; keine Datenbankoperation
BE-U-43	Aufgabe anlegen: Datenbankfehler tritt auf	SQL-Exception wird in fachliche Exception gewrappt; Controller-seitig wird Fehlerantwort zurückgegeben

5.1.3 Integrationstests

Begründung: Integrationstests prüfen das Zusammenspiel aller Backend-Schichten im vollständigen Spring-Kontext mit H2 In-Memory-Datenbank und MockMvc für HTTP-Endpunkt-Tests.

Namenskonvention (verbindlich):

- Unit-Test-Klassen: `*Test.java` → werden von Maven Surefire ausgeführt
- Integrationstest-Klassen: `*IT.java` → werden von Maven Failsafe ausgeführt (Phase integration-test und verify)

Diese Trennung ist verbindlich und beeinflusst die CI-Pipeline-Ausführung.

Testszenarien - Endpunkt: Aufgabe anlegen (POST /v1/tasks, /F050/):

ID	Szenario	Erwartetes Ergebnis
BE-I-01	Gültige Eingabe (Name >= 1 Zeichen, Dauer vorhanden)	HTTP 200/201; Task wird in der Datenbank persistiert; Response enthält erstellten Task
BE-I-02	Name zu kurz (< 1 Zeichen)	HTTP 400; Fehlermeldung im Response-Body
BE-I-03	Name zu lang (> 128 Zeichen)	HTTP 400; Fehlermeldung im Response-Body
BE-I-04	Dauer fehlt im Request-Body	HTTP 400; Fehlermeldung im Response-Body
BE-I-05	Leerer Request-Body	HTTP 400

Testszenarien - Endpunkt: Zeitslot anlegen (POST /v1/organizationslots, /MZ040/):

ID	Szenario	Erwartetes Ergebnis
BE-I-10	Gültige Eingabe (Start und Dauer vorhanden)	HTTP 200/201; Zeitslot wird persistiert
BE-I-11	Start fehlt	HTTP 400
BE-I-12	Dauer fehlt	HTTP 400

Testszenarien - Endpunkt: Planung auslösen (POST /v1/plan/, /F160/):

ID	Szenario	Erwartetes Ergebnis
BE-I-20	Tasks und Slots vorhanden; ausreichend Kapazität	HTTP 200; Scopes werden berechnet und in der Response zurückgegeben
BE-I-21	Tasks vorhanden, aber unzureichende Slot-Kapazität	HTTP 400; aussagekräftige Fehlermeldung im Response-Body
BE-I-22	Keine Tasks vorhanden	HTTP 200; leere Scope-Liste

5.2 Frontend

5.2.1 Komponenten-Unit-Tests

Begründung: Angular-Standalone-Components werden hinsichtlich ihrer TypeScript-Logik isoliert getestet. DOM-Testing (.html-Templates) wird im Rahmen dieses Projekts bewusst ausgelassen, da der Aufwand-Nutzen-Faktor für die Projektgröße und den Zeitrahmen nicht gerechtfertigt ist und Grenzen in der Testbibliothek bei komplexen Komponenten wie FullCalendar bestehen. Geprüft werden Zustandsübergänge, Methodenlogik und das korrekte Auslösen von Service-Aufrufen.

Testszenarien - Formularkomponenten (/F050/, /F080/):

ID	Szenario	Erwartetes Ergebnis
FE-U-01	Pflichtfelder fehlen (z. B. Titel leer)	Submit-Aktion wird blockiert; Fehler-State der Formularfelder ist aktiv
FE-U-02	Alle Pflichtfelder korrekt ausgefüllt	Submit-Methode löst Service-Aufruf aus
FE-U-03	Formular abbrechen (Cancel)	Kein Service-Aufruf; Formular-State wird zurückgesetzt

Testszenarien - Modal-Service:

ID	Szenario	Erwartetes Ergebnis
FE-U-10	Modal öffnen	open$-Observable emittiert true; Modal-Zustand ist offen
FE-U-11	Modal schließen	open$-Observable emittiert false; Modal-Zustand ist geschlossen

Testszenarien - Einstellungs-Store (/F130/):

ID	Szenario	Erwartetes Ergebnis
FE-U-20	Einstellung speichern	Wert ist danach über den Store abrufbar; Persistenz-Adapter wird aufgerufen
FE-U-21	Einstellung lesen nach initialem Laden	Wert entspricht dem gespeicherten Persistenzstand

5.2.2 Service-Unit-Tests

Begründung: Services stellen die zentrale Zustandsverwaltung und Backend-Kommunikation bereit. Sie werden isoliert mit gemockten Abhängigkeiten

getestet.

Testszenarien - Aufgaben-Service (/F050/, /F060/, /F070/):

ID	Szenario	Erwartetes Ergebnis
FE-U-30	Aufgabe zum State hinzufügen	tasks$-Observable enthält die neue Aufgabe
FE-U-31	Aufgabe aus State entfernen	tasks$-Observable enthält die Aufgabe nicht mehr
FE-U-32	Aufgabe aktualisieren	tasks$-Observable enthält die aktualisierte Version der Aufgabe
FE-U-33	Aufgabe als erledigt markieren (/F070/)	Aufgabe im State hat `isFinished = true`

Testszenarien - Authentifizierungs-Service (/F151/, /F152/):

ID	Szenario	Erwartetes Ergebnis
FE-U-40	Login-Methode wird aufgerufen	OAuthService.initCodeFlow() wird an den Identity Provider delegiert (gemockter OAuthService)
FE-U-41	Logout-Methode wird aufgerufen	OAuthService.logOut() wird aufgerufen; lokale Session wird beendet

5.2.3 Manuelle Tests

Entwickler testen UI-seitiges Verhalten (z. B. Kalenderdarstellung, Modalöffnung und -schließung, Filterinteraktionen, Responsiveness) manuell durch Browserbedienung. Manuelle Tests sind nicht Teil der automatisierten CI-Pipeline und werden ergänzend zu den automatisierten Tests durchgeführt. Für manuelle Tests gibt es kein festes Protokoll - Entwickler führen sie vor allem nach der Implementierung neuer Features

und vor Pull Requests durch.

5.3 Sicherheitsszenarien

Die folgenden Szenarien leiten sich aus den Anforderungen /K020/, /K021/, /QF010/ sowie dem Designentwurf ab. Sie sind bereits fachlich spezifiziert, können aber erst vollständig als automatisierte Tests implementiert werden, sobald die Authentifizierungspflicht im Backend aktiv erzwungen wird. Bis dahin dienen sie als Planungsgrundlage.

Testszenarien - Backend-Endpunktsicherheit (/K020/, /K021/):

ID	Szenario	Erwartetes Ergebnis
BE-S-01	Request ohne JWT-Token an einen geschützten Endpunkt	HTTP 401 Unauthorized
BE-S-02	Request mit abgelaufenem JWT-Token	HTTP 401 Unauthorized
BE-S-03	Request mit syntaktisch ungültigem JWT	HTTP 401 Unauthorized
BE-S-04	Request mit gültigem JWT; Benutzer noch nicht in der Datenbank	Account wird automatisch angelegt; Request wird normal verarbeitet (/F153/)
BE-S-05	Request mit gültigem JWT, der Organisationszugehörigkeit enthält; Account-Organisation-Verknüpfung fehlt	Verknüpfung wird automatisch angelegt; Request wird normal verarbeitet

Testszenarien - Account-Verknüpfung (/F030/):

ID	Szenario	Erwartetes Ergebnis
BE-S-10	Verknüpfungsanfrage mit E-Mail-Adresse, die keinem Account zugeordnet ist	Keine Bestätigungs-E-Mail wird versendet; Operation wird abgelehnt

BE-S-11	Verknüpfungsanfrage mit gültiger Ziel-E-Mail	Bestätigungs-E-Mail wird an die Ziel-Adresse gesendet; Identitäten werden erst nach Bestätigung zusammengeführt

5.4 Nicht behandelte Testarten

Frontend-Integrationstests: Angular-Integrationstests (Komponenten mit echten Services) werden aufgrund des begrenzten Zeitrahmens ausgelassen. Die wichtigsten Zustandsflüsse werden durch die Service-Unit-Tests (Abschnitt 5.2.2) bereits ausreichend auf Korrektheit geprüft.

End-to-End-Tests (E2E): E2E-Tests werden im Rahmen dieses Projekts bewusst ausgelassen. Der begrenzte Zeitrahmen erlaubt keinen vertretbaren Aufwand für den Auf- und Abbau einer vollständigen E2E-Testinfrastruktur (laufendes Backend, Browser-Automatisierung, stabile Testumgebung). Die kritischen Pfade des Systems sind durch Backend-Integrationstests und Frontend-Komponententests ausreichend abgedeckt.

6 Testwerkzeuge

6.1 Backend

Werkzeug	**Einsatzzweck**
JUnit 5 (via Spring Boot Starter Test)	Testframework für Unit- und Integrationstests
Mockito	Erstellung von Mocks und Stubs für Repository- und Service-Abhängigkeiten in Unit-Tests
MockMvc (@AutoConfigureMockMvc)	HTTP-Endpunkt-Tests ohne echten Netzwerk-Stack
Spring Boot Test (@SpringBootTest)	Starten des vollständigen Spring Application Context für Integrationstests

H2 (In-Memory, runtime-Scope)	Testdatenbank für Integrationstests; ersetzt MariaDB in der Testphase automatisch
Maven Surefire	Ausführung von *Test.java-Klassen (Unit Tests)
Maven Failsafe	Ausführung von *IT.java-Klassen (Integrationstests) in den Phasen integration-test und verify

6.2 Frontend

Werkzeug	**Einsatzzweck**
Vitest	Test-Runner; konfiguriert über angular.json und tsconfig.spec.json; stellt Globals (describe, it, expect, vi) ohne expliziten Import bereit
Angular TestBed / ComponentFixture	Konfiguration und Instanziierung von Angular-Komponenten und Services unter Test-Bedingungen
Vitest-Mock-API	Erstellung von Mocks und Spies für Service-Abhängigkeiten

6.3 Testautomatisierung / CI

GitHub Actions wird als CI/CD-Werkzeug zur automatisierten Ausführung der Tests eingesetzt.

- Tests werden bei Push auf den main-Branch sowie bei Pull Requests automatisch ausgeführt.
- Tests sind die erste Stage in der Pipeline: Schlagen sie fehl, werden nachfolgende Pipeline-Steps (z. B. Build, Deployment) nicht ausgeführt.
- Pull Requests können nicht gemerged werden, wenn Tests fehlschlagen (Branch-Schutzregel).
- Testergebnisse sind im jeweiligen GitHub-Actions-Pipeline-Run einsehbar.

7 Teststrategie

Das Projekt folgt dem Test-first-Ansatz: Tests werden vor oder parallel zur Implementierung der entsprechenden Funktionalität geschrieben. Dies stellt sicher, dass die Implementierung die spezifizierten Anforderungen erfüllt, und fördert ein testbares Design.

Trennung von Unit- und Integrationstests:

Tests werden klar in Unit-Tests (keine externen Abhängigkeiten, keine Datenbankzugriffe, kein Spring-Kontext) und Integrationstests (vollständiger Kontext, Datenbankzugriffe) getrennt. Die Trennung erfolgt konsequent über die Dateinamenkonvention des Maven-Build-Systems.

Entwickler-Verantwortung: Jeder Entwickler ist dafür verantwortlich, für neu implementierte oder geänderte Funktionen auch Tests zu schreiben bzw. bestehende Tests anzupassen. Diese Erwartung gilt auch für Bugfixes: Ein Bugfix soll idealerweise mit einem Regressionstest begleitet werden.

7.1 Mocking-Strategie

Backend:

- Repository-Abhängigkeiten werden in Service-Unit-Tests mit Mockito gemockt.
- Externe Dienste (Keycloak, SMTP) werden in Integrationstests nicht aufgerufen; JWT-Validierung ist im aktuellen Prototyp-Stand noch nicht erzwungen.
- Die H2 In-Memory-Datenbank ersetzt MariaDB automatisch in Integrationstests.

Frontend:

- Service-Abhängigkeiten werden in Komponenten-Unit-Tests mit vi.fn() / vi.spyOn() gemockt.
- OAuthService (aus angular-oauth2-oidc) wird in allen Tests, die den AuthService oder App-Komponente einbinden, als Mock bereitgestellt, da App beim Start auth.login() aufruft.
- Der SETTINGS_DATA_SOURCE-DI-Token muss in Tests, die den SettingsStore

einbinden, explizit als Mock bereitgestellt werden.
- HTTP-Calls werden gemockt (aktuell sind noch keine echten HTTP-Calls implementiert; sobald HTTP-Kommunikation eingeführt wird, wird HttpTestingController eingesetzt).
- Mocks werden zwischen Tests mit `vi.clearAllMocks()` zurückgesetzt.

7.2 Testdaten

Testdaten werden in der Regel direkt im Test definiert. Bei komplexeren Eingaben (z. B. größere Aufgabengraphen für Algorithmus-Tests) können Testdaten auch aus Dateien im Testressourcen-Verzeichnis geladen werden. Es gibt kein zentrales Testdaten-Repository. Die H2 In-Memory-Datenbank wird bei jedem Integrationstest-Lauf neu aufgebaut, sodass Tests nicht voneinander abhängen.

7.3 Testabdeckung

Zur Sicherstellung einer angemessenen strukturellen Testtiefe wird für automatisierte Tests projektweit eine Zweigüberdeckung von mindestens 75% angestrebt. Für das fachlich besonders kritische Algorithmus-Layer (CPM, topologische Sortierung, Planungsalgorithmus) wird eine Zweigüberdeckung von mindestens 90% angestrebt.

Die höhere Zielsetzung für das Algorithmus-Layer ist durch dessen zentrale fachliche Bedeutung und das erhöhte Risiko schwer erkennbarer Logikfehler begründet. Für weniger kritische Bereiche, insbesondere einfache Delegationslogik, Infrastruktur-Code oder UI-Hilfslogik, ist eine geringere Abdeckung im Einzelfall vertretbar, sofern die fachlich relevanten Normal-, Fehler- und Randfälle durch Tests abgesichert sind.

Die definierten Abdeckungsziele sind als Qualitätsziele für die Implementierungsphase zu verstehen. Eine begründete Unterschreitung ist in Ausnahmefällen zulässig, muss dann jedoch im Team nachvollziehbar begründet werden. Für das Algorithmus-Layer sind Abweichungen nur in fachlich begründeten Sonderfällen vertretbar.

8 Testumgebung

8.1 Lokale Entwicklungsumgebung

Entwickler führen Tests lokal vor größeren Änderungen und vor dem Erstellen von Pull Requests aus, um Fehler frühzeitig zu erkennen.

Aspekt	Detail
Backend-Tests	`./mvnw test` (Unit Tests via Surefire) / `./mvnw verify` (inkl. Integrationstests via Failsafe)
Frontend-Tests	`pnpm test` (→ ng test via Vitest)
Datenbank (Test)	H2 In-Memory (automatisch, kein Setup erforderlich)
Externe Dienste	Werden gemockt; kein Keycloak oder SMTP erforderlich

8.2 CI-Umgebung (GitHub Actions)

Aspekt	Detail
Trigger	Push auf main, Pull Requests
Backend	Maven-Build mit `mvn verify` (führt Unit- und Integrationstests aus)
Frontend	`pnpm test` in entsprechender Node.js-Umgebung
Artefakte	Testergebnisse werden als Pipeline-Logs gespeichert und sind im GitHub-Actions-Run einsehbar

9 Testkriterien

9.1 Eintrittskriterien

Tests werden für eine Komponente oder ein Feature geschrieben, sobald:

- Die funktionale Anforderung im Pflichtenheft spezifiziert ist (für spezifikationsbasierte Tests).
- Der Code, der getestet werden soll, in einer stabilen Zwischenversion vorliegt (für implementierungsnahe Tests).

9.2 Abbruchkriterien

Die Testausführung in der CI-Pipeline wird abgebrochen und das Merge in main wird blockiert, wenn:

- Ein oder mehr Tests fehlschlagen.
- Der Build-Prozess nicht erfolgreich abgeschlossen werden kann.

9.3 Erfolgskriterien / Abnahme

Ein Feature gilt als testmäßig abgenommen, wenn:

- Alle automatisierten Tests für das Feature erfolgreich durchlaufen.
- Die fachlichen Testszenarien aus Kapitel 5 für das betreffende Feature durch Tests abgedeckt sind.
- Die angestrebte Testabdeckung im betroffenen Modul erreicht ist oder eine begründete Abweichung dokumentiert wurde.
- Kein bekannter Fehler in den Kernfunktionen (/F050/, /F060/, /F160/) existiert, der durch einen fehlschlagenden Test nicht erkannt werden würde.

10 Dokumentation und Testergebnisse

Testergebnisse werden nicht in separaten Testberichten dokumentiert, sondern sind direkt im jeweiligen GitHub-Actions-Pipeline-Run einsehbar. Jeder Run enthält:

- Protokoll der ausgeführten Tests (bestanden / fehlgeschlagen)
- Konsolenausgabe mit Details zu fehlgeschlagenen Tests
- soweit verfügbar Coverage-Reports zur Beobachtung der Testabdeckung

Fehlgeschlagene Tests in einem Pull Request führen zu einem sichtbaren Status-Check-Fehler auf dem PR und blockieren das Mergen.

Testfälle selbst sind im Quellcode der Testklassen dokumentiert (Testmethoden mit sprechenden Namen). Übergreifende Testszenarien sind in diesem Dokument (Kapitel 5) beschrieben.

11 Verantwortlichkeiten

Person	**Aufgabe**
Elias Kerlin	Gesamtverantwortung für Tests: Pflege des Testkonzepts, Sicherstellung der Teststrategie, Eskalation bei systemischen Testproblemen
Alle Entwickler	Tests für neu implementierte oder geänderte Funktionen schreiben bzw. anpassen; Tests vor PRs lokal ausführen
PR-Reviewer	Überprüfung, ob ausreichende Testabdeckung für die geänderten Funktionen vorhanden ist; Fehlschlagende Tests sind ein Merge-Blocker

12 Risiken

#	Risiko	Wahrscheinlichkeit	Auswirkung	Maßnahme
R-01	Testabdeckung: Der begrenzte Zeitrahmen erhöht das Risiko, dass Tests unter Termindruck weggelassen oder oberflächlich geschrieben werden. Besonders Randfälle und Fehlerpfade sind gefährdet.	Mittel	Hoch	Testziele (Kap. 4) priorisieren explizit: Algorithmus-Layer und Backend-Integrationstests haben höchste Priorität.
R-02	Sicherheitsszenarien können erst vollständig getestet werden, wenn die Authentifizierungspflicht aktiv erzwungen wird. Solange alle Endpunkte permitAll() sind, bleibt die Sicherheitsschicht faktisch untestet.	Hoch	Hoch	Szenarien sind in Abschnitt 5.3 bereits spezifiziert. Sobald Auth erzwungen wird, müssen die dortigen Testszenarien als erste Priorität implementiert werden.

R-03	Anforderungsänderungen invalidieren bestehende Tests: Änderungen am Datenmodell, an der API-Signatur oder am Algorithmus können dazu führen, dass bestehende Tests veralten oder falsch-positiv grün bleiben.	Mittel	Mittel	Testklassen müssen bei jeder inhaltlichen Anforderungs- oder Designänderung aktiv mitgepflegt werden. Der PR-Reviewer trägt Mitverantwortung dafür, dass betroffene Tests angepasst werden.
R-04	Algorithmuskorrektheit ist schwer vollständig verifizierbar: Der Backtracking-Planungsalgorithmus hat eine exponentielle Anzahl möglicher Kombinationen. Nicht alle Randfälle lassen sich im Voraus als Testszenarien identifizieren.	Mittel	Mittel	Fokus auf repräsentative Äquivalenzklassen (Normalfall, unlösbar, Backtracking-notwendig). Bei auftretenden Bugs werden Regressionstests ergänzt.